

A AURORA ESTABELECE OS  
PRIMEIROS SISTEMAS DA COLÔNIA

AS ESTRUTURAS  
AVANÇADAS QUE  
RASTREIAM  
RECURSOS E  
INFORMAÇÕES  
DA COLÔNIA

4

## LISTA DE FIGURAS

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| Figura 1 - Resultado da árvore gerada pelo BST .....                                                | 10 |
| Figura 2 - Exemplo de BST desbalanceada .....                                                       | 11 |
| Figura 3 - Ordem rotacionada para evitar o desbalanceamento utilizando a árvore AVL .....           | 16 |
| Figura 4 - Exemplo de árvore <i>Red-Black</i> : Forma incorreta (esq.) e forma correta (dir.) ..... | 17 |
| Figura 5 - Funcionamento da função <i>hash</i> .....                                                | 27 |



## LISTA DE CÓDIGOS-FONTE

|                                                                                                   |    |
|---------------------------------------------------------------------------------------------------|----|
| Código-fonte 1 – Exemplo de código da árvore binária em um programa em C .....                    | 7  |
| Código-fonte 2 – Exemplo de código da árvore binária - BST em um programa em C .....              | 9  |
| Código-fonte 3 – Exemplo de código da árvore AVL em um programa em C .....                        | 15 |
| Código-fonte 4 – Exemplo de código da inserção em <i>Red-Black Tree</i> em um programa em C ..... | 23 |
| Código-fonte 5 – Exemplo de código de <i>hash</i> em um programa em C .....                       | 29 |

EMANIP

## SUMÁRIO

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| ÁRVORES E TABELAS HASH .....                                              | 5  |
| 1 INTRODUÇÃO .....                                                        | 5  |
| 2 FUNDAMENTOS DAS ÁRVORES.....                                            | 6  |
| 2.1 O que é uma árvore? .....                                             | 6  |
| 2.2 Árvore binária .....                                                  | 7  |
| 2.3 Árvores binárias de busca (BST – <i>Binary Search Trees</i> ) .....   | 8  |
| 2.3.1 Exemplo prático do funcionamento da BST .....                       | 9  |
| 2.3.2 O problema da BST desbalanceada .....                               | 10 |
| 2.4 Árvores balanceadas: A motivação e o surgimento das árvores AVL ..... | 11 |
| 2.4.1 O fator de balanceamento ( <i>Balance Factor</i> – BF) .....        | 12 |
| 2.4.2 Intuição com um exemplo prático .....                               | 13 |
| 2.4.3 Exemplo prático .....                                               | 15 |
| 2.5 <i>Red-Black Trees</i> (Árvores Rubro-Negras).....                    | 16 |
| 2.5.1 Propriedades fundamentais.....                                      | 17 |
| 2.5.2 Operações em <i>Red-Black Trees</i> .....                           | 18 |
| 2.5.3 Analogia prática para compreensão .....                             | 23 |
| 2.5.4 Comparação entre AVL <i>Trees</i> e <i>Red-Black Trees</i> .....    | 24 |
| 3 TABELAS <i>HASH</i> .....                                               | 25 |
| 3.1 Funcionamento conceitual.....                                         | 26 |
| 3.2 Função <i>hash</i> .....                                              | 26 |
| 3.3 Colisões.....                                                         | 27 |
| 3.4 Analogia para compreensão da tabela <i>hash</i> .....                 | 27 |
| 4 CONCLUSÃO.....                                                          | 30 |
| REFERÊNCIAS.....                                                          | 31 |

# ÁRVORES E TABELAS HASH

## 1 INTRODUÇÃO

Após estudarmos os algoritmos de busca e ordenação, e compreendermos como os dados organizados influenciam diretamente a eficiência computacional, avançamos agora para estruturas capazes de oferecer desempenho superior aos métodos lineares tradicionais: árvores e tabelas *hash*.

Enquanto vetores e listas apresentam acessos lineares ou, na melhor das situações, dependem de ordenação para permitir buscas logarítmicas, as estruturas estudadas neste capítulo introduzem novos paradigmas de organização da informação, baseados em:

- Hierarquia → nas árvores, que distribuem elementos em níveis;
- Dispersão com função *hash* → nas tabelas *hash*, que mapeiam chaves para posições de armazenamento.

Ambas têm um papel central no desenvolvimento de sistemas modernos. Segundo Knuth (1998), árvores binárias de busca bem balanceadas permitem localizar, inserir e remover elementos em  $O(\log n)$ , superando as limitações das listas sequenciais. Já as tabelas *hash*, quando bem projetadas, atingem desempenho médio  $O(1)$ , oferecendo um dos acessos mais rápidos possíveis em estruturas de dados.

Além de sua relevância teórica, tais estruturas constituem os pilares de componentes críticos como:

- Bancos de dados e sistemas de indexação;
- Compiladores e analisadores sintáticos;
- Sistemas operacionais;
- Caches e estruturas de memória;
- Dicionários, *sets* e *maps* em diversas linguagens;
- Sistemas embarcados sensíveis à performance.

Este material, portanto, aprofunda não apenas a modelagem dessas estruturas, mas também o entendimento matemático e computacional que sustenta sua eficiência, preparando você para temas avançados como árvores balanceadas, *hashing* criptográfico e algoritmos de compressão.

## 2 FUNDAMENTOS DAS ÁRVORES

Segundo Cormen *et al.* (2009), a importância das árvores decorre tanto de sua flexibilidade estrutural quanto de suas aplicações práticas, que vão desde bancos de dados e mecanismos de indexação até compressão de dados, inteligência artificial e sistemas operacionais. A forma como as árvores distribuem seus nós permite que operações essenciais sejam executadas, em média, em tempo  $O(\log n)$ , tornando-as uma das estruturas mais eficientes da computação moderna.

Aqui, exploraremos os conceitos fundamentais que definem uma árvore: raízes, nós, folhas, grau, profundidade, altura e subárvores. Em seguida, avançaremos para árvores binárias, árvores binárias de busca e propriedades que tornam essas estruturas tão poderosas para organizar e manipular dados.

Assim como a ordenação serviu de base para buscas mais rápidas, as árvores servirão agora como base para estruturas ainda mais sofisticadas, incluindo as tabelas *hash*, que representam outra abordagem de acesso eficiente a dados.

Com esse panorama estabelecido, damos início ao estudo formal das árvores e às operações que permitirão compreender seu papel fundamental nos algoritmos e sistemas contemporâneos.

### 2.1 O que é uma árvore?

Uma **árvore** é uma estrutura de dados hierárquica composta por elementos chamados **nós**, conectados através de relações do tipo pai-filho. Ao contrário de vetores e listas, que são lineares, as árvores permitem organização ramificada, tornando possível reduzir drasticamente o número de comparações necessárias para localizar elementos. Formalmente, uma árvore é um conjunto finito de nós, que é vazio ou composto por um nó especial chamado **raiz**, ligado a zero ou mais subárvores disjuntas.

Características essenciais:

- Possui **raiz**, que é o primeiro nó;
- Cada nó pode ter **0 ou mais filhos**;
- Não existem ciclos;
- Caminhos são unidirecionais e hierárquicos.

Entre as aplicações típicas dessas árvores, podemos citar:

- Hierarquia de arquivos em sistemas operacionais;
- Representação de expressões matemáticas;
- Árvores de decisão em *machine learning*;
- Índices *B-Tree* em bancos de dados;
- Estruturas como *heaps* (vistos em ordenação).

## 2.2 Árvore binária

A árvore binária é um tipo particular de árvore em que cada nó possui até dois filhos: esquerdo e direito. Ela é fundamental porque serve de base para estruturas mais avançadas. Abaixo temos uma representação da árvore binária na linguagem C.

```
typedef struct no {  
    int valor;  
    struct no *esq;  
    struct no *dir;  
} No;
```

Código-fonte 1 – Exemplo de código da árvore binária em um programa em C  
Fonte: Elaborado pelo autor (2025)

Cada estrutura de nó contém:

- O valor armazenado;
- Um ponteiro para o **filho esquerdo**;
- Um ponteiro para o **filho direito**.

## 2.3 Árvores binárias de busca (BST – *Binary Search Trees*)

Uma BST aplica às árvores o mesmo princípio da busca binária: dividir o espaço de busca a cada comparação. A regra é simples:

- Valores menores ficam à esquerda do nó;
- Valores maiores ficam à direita do nó.

Isso significa que, ao buscar um elemento, não precisamos verificar cada valor; basta, a cada passo, escolher entre esquerda ou direita, reduzindo a busca pela metade.

Colocando isso para o nosso dia-a-dia, imagine uma grande biblioteca com milhares de livros. Se todos os livros estivessem empilhados em um único monte, seria necessário examinar um por um até encontrar o desejado, exatamente como uma busca linear.

Agora, imagine que os livros estão organizados em uma estrutura de decisões:

- Na entrada, há um atendente que pergunta: “O título que você procura começa com A–M ou N–Z?” Se for A–M, você vai para a ala esquerda; se for N–Z, para a ala direita;
- Entrando na ala esquerda, outro atendente pergunta: “Entre A–F ou G–M?” E assim por diante...

Cada pergunta elimina metade da biblioteca como possibilidade, e esse é exatamente o comportamento de uma BST bem organizada.

Portanto, a BST é como uma biblioteca que ajuda você a caminhar por decisões sucessivas, filtrando rapidamente o que não interessa.

O código a seguir apresenta a implementação da árvore BST em C.

```
// Estrutura de um nó da árvore
typedef struct No {
    int valor; // Valor armazenado no nó struct
    No *esq; // Ponteiro para o filho esquerdo struct
    No *dir; // Ponteiro para o filho direito
```



```
    } No;

// Função de inserção em uma Árvore de Busca Binária (BST)

No* inserir(No* raiz, int valor) {
// Caso base: se a árvore está vazia ou chegamos a uma folha
    if (raiz == NULL) {
        No* novo = malloc(sizeof(No)); // Aloca novo nó
        novo->valor = valor;           // Define o valor do nó
        novo->esq = NULL;               // Não possui filhos
        novo->dir = NULL;               // Não possui filhos
        return novo; // Retorna o novo nó como parte da árvore
    }

// Caso recursivo: decidir se vamos para a esquerda ou direita

    if (valor < raiz->valor) {
// Se o valor for menor, insere na subárvore esquerda
        raiz->esq = inserir(raiz->esq, valor);
    }
    else if (valor > raiz->valor) {
// Se o valor for maior, insere na subárvore direita
        raiz->dir = inserir(raiz->dir, valor);
    }

// Se valor for igual, não inserimos (não permitimos duplicatas na BST)
// Retornamos a própria raiz sem modificar
    return raiz; // Retorna a raiz atualizada para manter a estrutura da árvore
}
```

Código-fonte 2 – Exemplo de código da árvore binária - BST em um programa em C  
Fonte: Elaborado pelo autor (2025)

### 2.3.1 Exemplo prático do funcionamento da BST

Considere a inserção na ordem:

**30, 20, 40, 10, 25**

Etapas a etapa:

1. **30** → árvore vazia → vira raiz;
2. **20** → menor que 30 → vai para a esquerda;

3. **40** → maior que 30 → vai para a direita;
4. **10** → menor que 30 → vai para a esquerda e menor que 20 → vai para a esquerda;
5. **25** → menor que 30 → esquerda e maior que 20 → direita.

Árvore resultante:

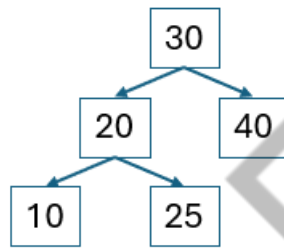


Figura 1 - Resultado da árvore gerada pelo BST  
Fonte: Elaborada pelo autor (2025)

A busca segue comparações dirigidas:

- **Buscar 25:**  $25 < 30$  → esquerda;  $25 > 20$  → direita → encontramos.

### 2.3.2 O problema da BST desbalanceada

A eficiência de uma BST depende de seu **equilíbrio**. Idealmente, sua altura é proporcional a  $\log_2(n)$ . Entretanto, isso **pode piorar drasticamente** dependendo da ordem de inserção.

Exemplo de inserção ruim:

**10, 20, 30, 40, 50**

Cada novo valor é maior que o anterior → sempre vai para a direita.

Árvore resultante:

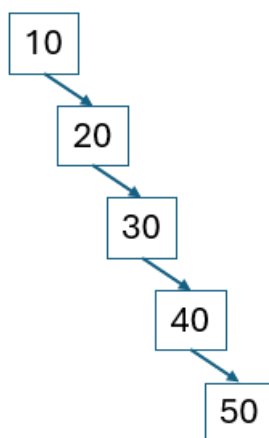


Figura 2 - Exemplo de BST desbalanceada  
Fonte: Elaborada pelo autor (2025)

Esse exemplo deixa a BST com comportamento idêntico a uma **lista encadeada**, com altura **n**. Consequências:

- Busca:  $O(n)$ ;
- Inserção:  $O(n)$ ;
- Remoção:  $O(n)$ .

Desse modo, perde-se totalmente o benefício da árvore.

Portanto, a BST funciona perfeitamente quando balanceada. Mas, quando desbalanceada, sua performance cai drasticamente. Isso motivará, a seguir, a introdução de árvores auto-balanceadas, como AVL, *Red-Black* e tabelas *hash*, que resolvem ou evitam tais limitações estruturais.

## 2.4 Árvores balanceadas: A motivação e o surgimento das árvores AVL

Após compreender o funcionamento das árvores binárias de busca e observar como seu desempenho depende diretamente do equilíbrio estrutural, torna-se evidente a necessidade de um mecanismo capaz de manter a árvore balanceada automaticamente, evitando os cenários patológicos em que a altura cresce linearmente ( $O(n)$ ), prejudicando todas as operações fundamentais (busca, inserção e remoção).

É nesse contexto que surgem as árvores AVL, nome derivado de seus criadores Adelson-Velsky e Landis (1962). Elas foram as primeiras estruturas de

árvore binária de busca a introduzir um critério formal de balanceamento dinâmico, garantindo que a altura da árvore se mantenha sempre proporcional a  $\log_2(n)$ . Isso preserva a eficiência das operações fundamentais:

- Busca:  $O(\log n)$ ;
- Inserção:  $O(\log n)$ ;
- Remoção:  $O(\log n)$ .

Em contraste com a BST tradicional, a AVL evita o crescimento assimétrico por meio da verificação contínua do fator de balanceamento de cada nó, aplicando rotações sempre que necessário.

Após uma inserção, quatro situações podem ocorrer:

1. Rotação simples à direita (*Right Rotation*);
2. Rotação simples à esquerda (*Left Rotation*);
3. Rotação dupla direita-esquerda (*Right-Left*);
4. Rotação dupla esquerda-direita (*Left-Right*).

Essas rotações reorganizam a subárvore localmente, mantendo a propriedade da BST intacta, enquanto recuperam a altura ideal.

#### 2.4.1 O fator de balanceamento (*Balance Factor* – BF)

O fator de balanceamento de um nó é definido como:

$$BF(n) = altura(sub\acute{a}rvore\_esquerda) - altura(sub\acute{a}rvore\_direita)$$

Em uma árvore AVL, para todo nó:

$$-1 \leq BF(n) \leq 1$$

Se essa condição for violada após uma inserção ou remoção, a árvore executa **rotações** que restauram o equilíbrio sem comprometer as propriedades da BST.

### 2.4.2 Intuição com um exemplo prático

Imagine uma fila de pessoas esperando atendimento em uma agência bancária. Se a fila estiver equilibrada entre guichês (cada lado atendendo quantidades semelhantes), o tempo médio de espera é baixo. Mas, se todos os clientes forem enviados sempre para o lado direito, ignorando o guichê à esquerda, a fila direita crescerá indefinidamente:

- Lado esquerdo → poucas pessoas;
- Lado direito → fila enorme.

Desse modo, o atendimento deixa de ser eficiente.

Nas BST's, ocorre o mesmo: se você insere valores sempre crescentes (ex.: 1, 2, 3, 4, 5), todos os nós ficam à direita → a árvore se transforma em uma lista → busca  $O(n)$ .

A árvore AVL funciona como um "gerente da agência" que redistribui clientes sempre que acontece um desequilíbrio.

A seguir temos um exemplo de implementação simplificada de inserção em AVL:

```
#include <stdio.h>
#include <stdlib.h>

// Estrutura básica do nó da AVL
typedef struct Node {
    int valor;
    struct Node *esq;
    struct Node *dir;
    int altura;
} Node;

// Função para obter a altura de um nó
int altura(Node *n) {
    return (n == NULL) ? 0 : n->altura;
}

// Atualiza a altura após modificações
int atualizarAltura(Node *n) {
    int h_esq = altura(n->esq);
    int h_dir = altura(n->dir);
    return (h_esq > h_dir ? h_esq : h_dir) + 1;
```

```
}

// Calcula fator de balanceamento
int fatorBalanceamento(Node *n) {
    return altura(n->esq) - altura(n->dir);
}

// Rotação simples à direita
Node* rotacaoDireita(Node *y) {
    Node *x = y->esq;
    Node *T2 = x->dir;

    // Rotação propriamente dita
    x->dir = y;
    y->esq = T2;

    // Atualiza alturas
    y->altura = atualizarAltura(y);
    x->altura = atualizarAltura(x);

    return x; // nova raiz
}

// Rotação simples à esquerda
Node* rotacaoEsquerda(Node *x) {
    Node *y = x->dir;
    Node *T2 = y->esq;

    // Rotação
    y->esq = x;
    x->dir = T2;

    // Atualiza alturas
    x->altura = atualizarAltura(x);
    y->altura = atualizarAltura(y);

    return y; // nova raiz
}

// Criação de um novo nó
Node* novoNo(int valor) {
    Node *n = malloc(sizeof(Node));
    n->valor = valor;
    n->esq = n->dir = NULL;
    n->altura = 1;
    return n;
}

// Inserção na árvore AVL
Node* inserir(Node *raiz, int valor) {
    // Inserção normal de BST
```

```
if (raiz == NULL)
    return novoNo(valor);

if (valor < raiz->valor)
    raiz->esq = inserir(raiz->esq, valor);
else if (valor > raiz->valor)
    raiz->dir = inserir(raiz->dir, valor);
else
    return raiz; // valores duplicados não são inseridos

// Atualiza altura
raiz->altura = atualizarAltura(raiz);

// Obtém fator de balanceamento
int bf = fatorBalanceamento(raiz);

// Caso 1: desbalanceado à esquerda (Left-Left)
if (bf > 1 && valor < raiz->esq->valor)
    return rotacaoDireita(raiz);

// Caso 2: desbalanceado à direita (Right-Right)
if (bf < -1 && valor > raiz->dir->valor)
    return rotacaoEsquerda(raiz);

// Caso 3: Left-Right
if (bf > 1 && valor > raiz->esq->valor) {
    raiz->esq = rotacaoEsquerda(raiz->esq);
    return rotacaoDireita(raiz);
}

// Caso 4: Right-Left
if (bf < -1 && valor < raiz->dir->valor) {
    raiz->dir = rotacaoDireita(raiz->dir);
    return rotacaoEsquerda(raiz);
}

return raiz; // nenhuma rotação necessária
}
```

Código-fonte 3 – Exemplo de código da árvore AVL em um programa em C  
Fonte: Elaborado pelo autor (2025)

### 2.4.3 Exemplo prático

Considere a inserção na ordem:

**30, 20, 10**

Etapa a etapa:

- Inserir 30 → árvore só tem um nó;

- Inserir 20 → fica à esquerda → ainda balanceada;
- Inserir 10 → gera desbalanceamento tipo *Left-Left*.

A rotação à direita reorganiza a estrutura (em que cada bloco numérico representa uma unidade de altura, ou seja, na figura “Ordem rotacionada para evitar o desbalanceamento utilizando a árvore AVL”, a altura da ordem à esquerda é igual a 3), fazendo com que a altura da árvore volte a ser logarítmica (direita).

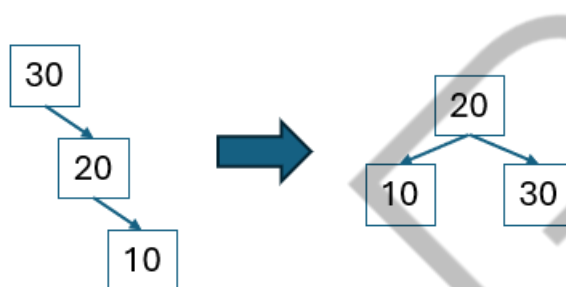


Figura 3 - Ordem rotacionada para evitar o desbalanceamento utilizando a árvore AVL  
Fonte: Elaborada pelo autor (2025)

## 2.5 Red-Black Trees (Árvores Rubro-Negras)

As *Red-Black Trees* representam uma das formas mais importantes de árvores de busca binária auto-balanceadas em ciência da computação. Elas foram desenvolvidas a partir da necessidade de manter o desempenho eficiente de operações básicas, como busca, inserção e remoção, mesmo em cenários adversos, nos quais uma árvore de busca binária simples poderia degenerar e perder sua eficiência (Cormen; Leiserson; Rivest; Stein, 2009; Sedgewick; Wayne, 2011).

Em árvores de busca binária comuns, quando dados são inseridos em ordem crescente ou decrescente, a estrutura pode se tornar “esparsa”, assemelhando-se a uma lista ligada, atingindo altura  $O(n)$ , e fazendo com que operações como busca, inserção e remoção tenham complexidade linear no pior caso. A motivação por trás das *Red-Black Trees* é justamente evitar essa degeneração.

Elas, portanto, resolvem esse problema ao impor restrições estruturais extras que limitam a altura da árvore. Isso ocorre associando a cada nó um atributo de cor (vermelho ou preto) e estabelecendo regras que garantem que a árvore permaneça aproximadamente balanceada após cada operação de modificação, preservando assim a performance logarítmica (Geeksforgeeks, 2025).



### 2.5.1 Propriedades fundamentais

Uma *Red-Black Tree* é uma árvore binária de busca em que cada nó possui um atributo de cor (vermelho ou preto). A estrutura deve obedecer a cinco propriedades essenciais:

1. **Propriedade da cor do nó:** cada nó é vermelho ou preto;
2. **Propriedade da raiz:** a raiz da árvore é sempre preta;
3. **Propriedade dos nós vermelhos:** um nó vermelho nunca pode ter um filho vermelho, ou seja, não existem dois nós vermelhos consecutivos em qualquer caminho da raiz até uma folha;
4. **Propriedade do caminho preto:** todo caminho de um nó até as suas folhas (nós nulos ou NIL, de Nillion, rede descentralizada) possui o mesmo número de nós pretos, um valor conhecido como *black-height*;
5. **Propriedade das folhas:** todas as folhas (nós nulos) são consideradas pretas (Geeksforgeeks, 2025).

Essas restrições evitam que a árvore se torne muito alta, mantendo seu crescimento dentro de  $O(\log n)$ , em que  $n$  representa o número de nós. Isso garante que as operações básicas sejam executadas em tempo logarítmico no pior caso (Cormen; Leiserson; Rivest; Stein, 2009; Sedgewick; Wayne, 2011).

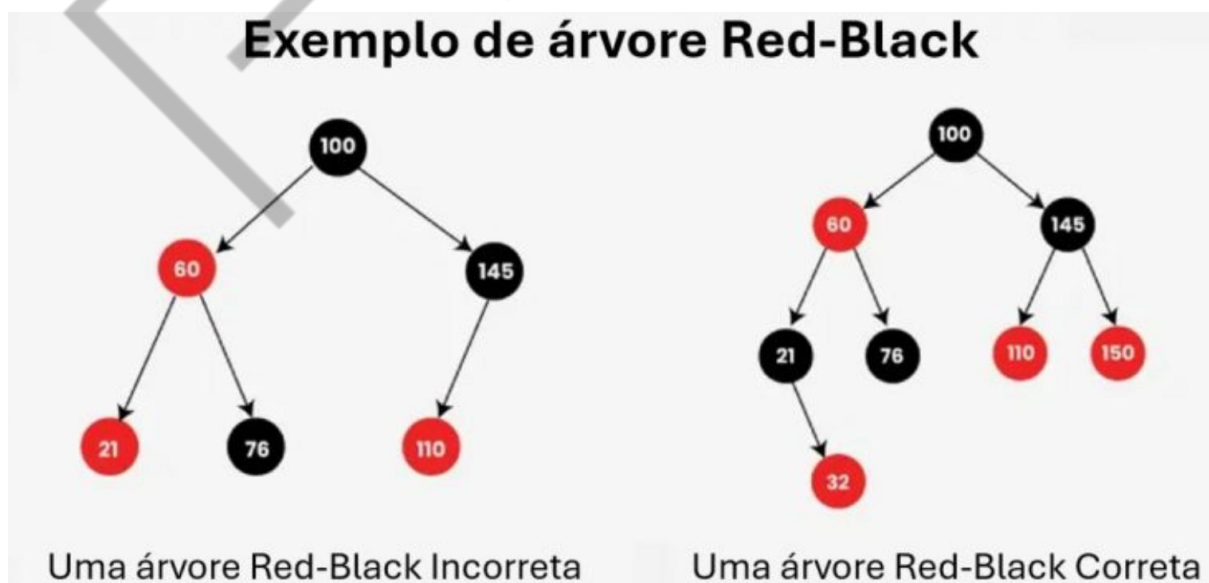


Figura 4 - Exemplo de árvore *Red-Black*: Forma incorreta (esq.) e forma correta (dir.)

Fonte: Traduzido de Geeksforgeeks (2025)

### 2.5.2 Operações em *Red-Black Trees*

As operações em *Red-Black Trees* seguem os princípios das árvores binárias de busca, mas, após uma modificação (inserção ou remoção), a estrutura pode violar alguma das propriedades de cor. Quando isso ocorre, são aplicadas rotações e recolorações para restaurar as propriedades originais sem comprometer a ordem dos elementos nem a eficiência do algoritmo (Geeksforgeeks, 2025).

- **Busca:** semelhante à busca em uma árvore binária de busca tradicional, atravessando a árvore da raiz até encontrar o valor ou confirmar sua ausência;
- **Inserção:** insere um novo nó como em uma BST simples, colorido inicialmente de vermelho. Depois a árvore é reequilibrada através de rotações e recolorações para preservar as propriedades de cor;
- **Remoção:** envolve passos mais complexos de balanceamento, pois uma remoção pode levar a violações adicionais. Aplicam-se casos específicos de rotações e recolorações para restabelecer propriedades (Geeksforgeeks, 2025).

As rotações (simples ou duplas) são operações locais que preservam a ordem da árvore, mas ajustam a estrutura para manter a altura balanceada (Cormen; Leiserson; Rivest; Stein, 2009).

Além disso, *Red-Black Trees* são particularmente valiosas quando é necessário garantir desempenho estável mesmo no pior caso, como em:

- Mapas e conjuntos ordenados: implementações de bibliotecas padrão, como `std::map` em C++ ou *TreeMap* em Java, usam *Red-Black Trees* para operações  $O(\log n)$ ;
- Sistemas operacionais e bancos de dados: onde eficiência e previsibilidade são essenciais para operações de armazenamento e recuperação de dados;
- Cenários com muitas inserções e remoções dinâmicas: a manutenção automática do balanceamento reduz a necessidade de reestruturações manuais (Geeksforgeeks, 2025).

Em comparação com uma BST desbalanceada, em que o pior caso pode ser tão ruim quanto  $O(n)$ , *Red-Black Trees* fornecem garantias assintóticas mais fortes e previsíveis (Cormen; Leiserson; Rivest; Stein, 2009).

O código a seguir concentra-se na lógica de inserção e no balanceamento via rotações e recoloração, seguindo a ideia discutida nas fontes de inserção e *fixup* de *Red-Black Trees* (similar a algoritmo RB-INSERT-FIXUP).

```
#include <stdio.h>
#include <stdlib.h>

// -----
// Definições de cor (0 = PRETO, 1 = VERMELHO)
// -----
#define PRETO 0
#define VERMELHO 1

// -----
// Estrutura de um nó da Red-Black Tree
// -----
typedef struct RBNode {
    int key;                // valor armazenado
    int color;              // cor do nó (VERMELHO ou
PRETO)
    struct RBNode *left;    // filho à esquerda
    struct RBNode *right;   // filho à direita
    struct RBNode *parent;  // pai do nó
} RBNode;

// Ponteiro global para o nó NIL (folhas sentinela)
RBNode *NIL;

// Ponteiro global para a raiz da árvore
RBNode *root;

// -----
// Função para criar um novo nó vermelho
// -----
RBNode* createNode(int key) {
    RBNode* node = (RBNode*)malloc(sizeof(RBNode));
    node->key = key;                // atribui valor
    node->color = VERMELHO;         // novos nós são inseridos como
vermelhos
    node->left = NULL;
    node->right = NULL;
    node->parent = NULL;
    return node;
}
```

```
// -----
// Rotação à esquerda
// Preenche os ponteiros para reorganizar subárvore
// -----
void leftRotate(RBNode *x) {
    RBNode *y = x->right;          // y será novo topo da subárvore
    x->right = y->left;              // ajustar filho esquerdo de y
    para x->right
        if (y->left != NIL) {
            y->left->parent = x;      // atualiza ponteiro
        }
    y->parent = x->parent;           // y agora aponta para o pai
    anterior de x
    if (x->parent == NIL) {
        root = y;
    }
    else if (x == x->parent->left) {
        x->parent->left = y;
    }
    else {
        x->parent->right = y;
    }
    y->left = x;                    // x torna-se filho esquerdo de
    y
    x->parent = y;
}

// -----
// Rotação à direita
// -----
void rightRotate(RBNode *x) {
    RBNode *y = x->left;
    x->left = y->right;
    if (y->right != NIL) {
        y->right->parent = x;
    }
    y->parent = x->parent;
    if (x->parent == NIL) {
        root = y;
    }
    else if (x == x->parent->right) {
        x->parent->right = y;
    }
    else {
        x->parent->left = y;
    }
    y->right = x;
    x->parent = y;
}
}
```

```
// -----  
// Função de ajuste após inserção (fix-up)  
// Reestabelece as propriedades da Red-Black Tree  
// -----  
void insertFixUp(RBNode *z) {  
    while (z->parent->color == VERMELHO) {  
        // Caso 1: pai é filho à esquerda do avô  
        if (z->parent == z->parent->parent->left) {  
            RBNode *y = z->parent->parent->right;  
            // Caso 1a: tio de z é vermelho  
            if (y->color == VERMELHO) {  
                z->parent->color = PRETO;  
                y->color = PRETO;  
                z->parent->parent->color = VERMELHO;  
                z = z->parent->parent;  
            }  
            else {  
                // Caso 1b: z é filho à direita  
                if (z == z->parent->right) {  
                    z = z->parent;  
                    leftRotate(z);  
                }  
                // Caso 1c: z é filho à esquerda  
                z->parent->color = PRETO;  
                z->parent->parent->color = VERMELHO;  
                rightRotate(z->parent->parent);  
            }  
        }  
        // Caso 2: pai é filho à direita do avô  
        else {  
            RBNode *y = z->parent->parent->left;  
            if (y->color == VERMELHO) {  
                z->parent->color = PRETO;  
                y->color = PRETO;  
                z->parent->parent->color = VERMELHO;  
                z = z->parent->parent;  
            }  
            else {  
                if (z == z->parent->left) {  
                    z = z->parent;  
                    rightRotate(z);  
                }  
                z->parent->color = PRETO;  
                z->parent->parent->color = VERMELHO;  
                leftRotate(z->parent->parent);  
            }  
        }  
        if (z == root) {  
            break;  
        }  
    }  
}
```

```
    root->color = PRETO; // raiz sempre preta
}

// -----
// Inserção de um novo nó
// -----
void insertNode(int key) {
    RBNode* z = createNode(key);

    // BST insertion
    RBNode* y = NIL;
    RBNode* x = root;
    while (x != NIL) {
        y = x;
        if (z->key < x->key) {
            x = x->left;
        } else {
            x = x->right;
        }
    }
    z->parent = y;
    if (y == NIL) {
        root = z;
    } else if (z->key < y->key) {
        y->left = z;
    } else {
        y->right = z;
    }

    z->left = NIL;
    z->right = NIL;
    z->color = VERMELHO;

    insertFixUp(z); // corrige violações
}

// -----
// Percurso em ordem (in-order)
/// mostra estrutura
// -----
void inorderTraversal(RBNode* node) {
    if (node != NIL) {
        inorderTraversal(node->left);
        printf("%d (%s)  ", node->key, node->color == VERMELHO
? "R" : "B");
        inorderTraversal(node->right);
    }
}

// -----
// Função principal (teste)
```

```
// -----  
int main() {  
    NIL = (RBNode*)malloc(sizeof(RBNode));  
    NIL->color = PRETO;  
    root = NIL;  
  
    insertNode(10);  
    insertNode(20);  
    insertNode(30);  
    insertNode(15);  
    insertNode(25);  
  
    printf("Inorder traversal of RB Tree:\n");  
    inorderTraversal(root);  
    return 0;  
}
```

Código-fonte 4 – Exemplo de código da inserção em *Red-Black Tree* em um programa em C  
Fonte: Elaborado pelo autor (2025)

### 2.5.3 Analogia prática para compreensão

Imagine que você está organizando fichas de papel com números em caixas empilhadas:

- Cada caixa representa um nó;
- Elas são organizadas em pilhas e subdivididas (como árvores);
- Cada caixa tem uma etiqueta de cor, vermelho ou preto, que controla o “nível” da caixa.

Quando você adiciona uma nova ficha:

1. Você a coloca no lugar correto como em uma pesquisa binária (lado esquerdo se menor, direito se maior);
2. Você pinta essa ficha de vermelho (porque tudo que entra é vermelho primeiro, como um alerta temporário);
3. Se a caixa acima também está vermelha, é alerta duplo → você precisa reorganizar (rotacionar ou recolorir) para manter o equilíbrio;
4. As regras de cores asseguram que o conjunto de caixas nunca fique “muito alto”, como se cada cor atuasse como um “nivelador” de altura.

Essa analogia ajuda a lembrar que:

- Buscar é como procurar uma gaveta específica;
- Inserir é como encaixar uma nova ficha e garantir que o conjunto de caixas continue equilibrado;
- Rotação + recoloração são como reorganizar as pilhas de caixas para que nada fique “pesado demais” de um lado.

Esse comportamento garante que, mesmo após várias inserções, a árvore continue com desempenho logarítmico, pois a altura nunca cresce de forma descontrolada (*balancing guarantees*) (Geeksforgeeks, 2025).

#### 2.5.4 Comparação entre AVL Trees e Red-Black Trees

As AVL Trees e as Red-Black Trees são duas das principais implementações de árvores binárias de busca auto-balanceadas, projetadas para evitar que uma árvore cresça desnecessariamente em altura e que operações como busca, inserção e remoção degradem para  $O(n)$ .

Ambas garantem complexidade  $O(\log n)$  para essas operações, mas diferem nos critérios de balanceamento: uma AVL Tree mantém um balanceamento rígido, exigindo que a diferença de altura entre as subárvores esquerda e direita de qualquer nó seja no máximo 1, o que resulta em uma árvore geralmente mais baixa e mais bem balanceada. Por outro lado, uma Red-Black Tree impõe um balanceamento mais relaxado por meio de regras de coloração dos nós (vermelho/preto), aceitando uma maior diferença de altura entre caminhos desde que as propriedades de cor sejam preservadas.

Como consequência dessa diferença, AVL Trees tendem a ter melhor desempenho de busca devido à menor altura da árvore, mas exigem mais rotações durante inserções e remoções para manter seu balanceamento rígido (AlgoCademy Blog, 2025; Fiveable.me, 2025).

Em termos de inserção e remoção, Red-Black Trees normalmente demandam menos rotações porque a condição de balanceamento é mais tolerante, já que as regras de coloração permitem que subárvores permaneçam ligeiramente desequilibradas antes de aplicar rotações, o que pode resultar em melhor desempenho em cenários com muitas modificações. Em contraste, AVL Trees,



embora mais equilibradas, podem sofrer com um custo de manutenção maior quando a estrutura sofre atualizações frequentes, o que pode ser menos eficiente em aplicações em que inserções e deleções ocorrem com alta frequência.

Por outro lado, quando comparada em termos de altura máxima, uma AVL *Tree* costuma possuir um limite de aproximadamente  $1,44 \log n$ , enquanto uma *Red-Black Tree* pode atingir até cerca de  $2 \log n$ , refletindo essa diferença de rigor no balanceamento (Algocademy Blog, 2025; Fiveable.me, 2025).

A escolha entre essas duas estruturas depende do padrão de uso: quando o objetivo principal é buscas rápidas e minimização da altura, AVL *Trees* são frequentemente vantajosas; já em situações com operações intensivas de inserção e remoção, a sobrecarga menor de balanceamento das *Red-Black Trees* pode torná-las mais apropriadas. Apesar dessas diferenças, ambas mantêm garantias assintóticas de  $O(\log n)$  para as principais operações, de modo que a seleção entre elas geralmente é baseada no perfil de acesso e atualização dos dados na aplicação específica (Algocademy Blog, 2025; Fiveable.me, 2025).

### 3 TABELAS HASH

As tabelas *hash* são estruturas de dados fundamentais projetadas para oferecer acesso extremamente rápido a informações por meio de uma chave. Diferentemente das árvores binárias de busca (como as ABB, AVL ou *Red-Black Trees*), em que a organização dos dados depende da ordem natural ou comparativa dos elementos, as tabelas *hash* organizam os dados com base em funções *hash*, que mapeiam diretamente uma chave para uma posição de armazenamento. A grande virtude dessas estruturas é que, sob condições adequadas, operações como inserção, busca e remoção podem ser realizadas em tempo amortizado constante,  $O(1)$ , independentemente do tamanho da coleção de dados.

A motivação por trás do uso de tabelas *hash* está em sistemas em que as operações de consulta são executadas de forma massiva e a velocidade importa mais do que manter uma ordenação completa dos dados. Exemplos práticos incluem: dicionários de palavras, caches de memória, implementações de *sets* e *maps* em linguagens de programação modernas e sistemas de indexação de bancos de dados.

### 3.1 Funcionamento conceitual

O funcionamento de uma tabela *hash* baseia-se em três componentes principais:

1. Uma função hash  $h(k)$ : responsável por transformar uma chave  $k$  (como uma *string* ou número) em um índice inteiro dentro de um espaço limitado (o tamanho da tabela);
2. O vetor de armazenamento: um *array* em que cada posição pode armazenar um item ou uma coleção de itens;
3. Estratégias de tratamento de colisões: métodos para resolver situações em que duas chaves diferentes produzem o mesmo índice *hash*.

### 3.2 Função *hash*

Uma função *hash* é uma função determinística que converte uma chave  $k$  em um valor inteiro  $h(k)$  dentro de um intervalo  $[0, m - 1]$ , em que  $m$  é o tamanho da tabela. Uma boa função *hash* deve distribuir uniformemente as chaves pelo espaço de índices para evitar agrupamentos excessivos.

Exemplo de função *hash* simples para inteiros:

$$h(k) = k \bmod m$$

Essa função pega o resto da divisão de  $k$  pelo tamanho da tabela  $m$ . A figura “Funcionamento da função *hash*” apresenta como fica o resultado após a função.

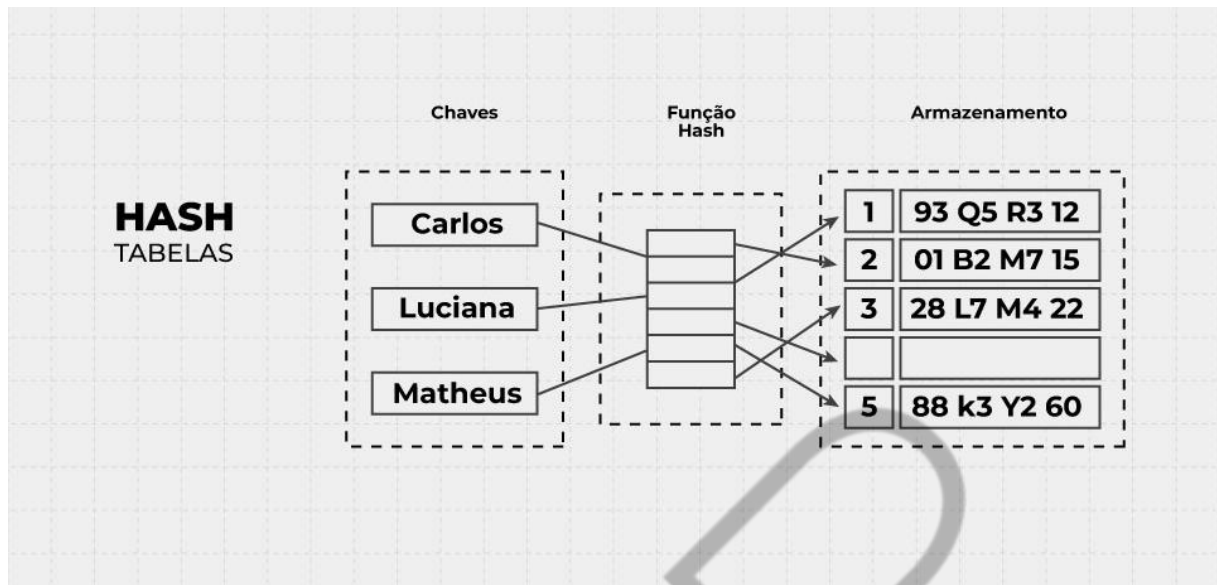


Figura 5 - Funcionamento da função *hash*  
Fonte: TERA (2025)

### 3.3 Colisões

Mesmo com funções *hash* eficientes, colisões podem ocorrer: duas chaves diferentes podem mapear para o mesmo índice. Existem duas estratégias principais para tratar colisões:

- Encadeamento separado (*separate chaining*): cada posição da tabela armazena uma lista (como uma lista ligada) de todos os elementos que mapeiam para aquele índice;
- Endereçamento aberto (*open addressing*): quando ocorre uma colisão, procura-se a próxima posição livre de acordo com uma sequência definida (por exemplo: linear, quadrática ou *hashing* duplo).

### 3.4 Analogia para compreensão da tabela *hash*

Imagine que você tem uma estante de caixas numeradas de 0 a  $m - 1$ . Quando você quer guardar um objeto, aplica uma “regra de divisão” sobre a etiqueta do objeto (por exemplo, pega o número e faz o módulo pelo número de caixas) para decidir em qual caixa ele deve ir. Às vezes, vários objetos podem ser enviados para a mesma caixa: isso é uma colisão. Para lidar com isso, você pode seguir escolher entre duas abordagens:

- **Encadeamento:** empilhar todos os objetos que vão para a mesma caixa em uma pequena pilha dentro dela;
- **Endereçamento aberto:** procurar a próxima caixa vazia e colocar o objeto ali.

Essa analogia ajuda a visualizar porque a tabela *hash* pode chegar perto de acesso direto (como se cada caixa fosse o número exato do objeto) e porque as colisões são inevitáveis, mas tratáveis.

A seguir temos um exemplo básico de implementação de uma tabela *hash* com encadeamento separado para valores inteiros:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 10

// Estrutura de nó para encadeamento separado
typedef struct Node {
    int key;
    struct Node* next;
} Node;

// Estrutura da tabela hash
Node* hashTable[TABLE_SIZE];

// Função hash simples: resto da divisão
int hashFunction(int key) {
    return key % TABLE_SIZE;
}

// Insere um valor na tabela
void insert(int key) {
    int index = hashFunction(key);
    Node* newNode = malloc(sizeof(Node));
    newNode->key = key;
    newNode->next = hashTable[index]; // insere no início da lista
    hashTable[index] = newNode;
}

// Busca um valor na tabela
int search(int key) {
    int index = hashFunction(key);
    Node* current = hashTable[index];
```

```
while (current != NULL) {
    if (current->key == key)
        return 1;
    current = current->next;
}
return 0;
}

// Remove um valor da tabela
void delete(int key) {
    int index = hashFunction(key);
    Node* current = hashTable[index];
    Node* prev = NULL;

    while (current != NULL && current->key != key) {
        prev = current;
        current = current->next;
    }
    if (current == NULL) return; // não encontrado

    if (prev == NULL) {
        hashTable[index] = current->next;
    } else {
        prev->next = current->next;
    }
    free(current);
}

int main() {
    memset(hashTable, 0, sizeof(hashTable));

    insert(15);
    insert(25);
    insert(35);

    printf("Busca 25: %s\n", search(25) ? "Encontrado" : "Nao
encontrado");
    delete(25);
    printf("Busca 25 apos remocao: %s\n", search(25) ?
"Encontrado" : "Nao encontrado");

    return 0;
}
```

Código-fonte 5 – Exemplo de código de *hash* em um programa em C  
Fonte: Elaborado pelo autor (2025)

## 4 CONCLUSÃO

Neste capítulo foram estudadas estruturas de dados não lineares fundamentais, com ênfase nas árvores de busca e nas tabelas *hash*, ambas projetadas para oferecer maior eficiência em operações de acesso, inserção e remoção quando comparadas às estruturas lineares. Inicialmente, foram apresentadas as árvores binárias de busca (BST), destacando seu funcionamento, suas vantagens conceituais e, principalmente, o problema do desbalanceamento, que pode degradar o desempenho para tempo linear no pior caso. Essa limitação motivou o estudo de árvores auto-balanceadas, que surgem como uma evolução natural das BST's.

Na sequência, foram analisadas as árvores AVL e as *Red-Black Trees*, duas abordagens distintas para garantir balanceamento e desempenho logarítmico. Enquanto as *AVL Trees* adotam um critério mais rígido de balanceamento, favorecendo buscas mais rápidas devido à menor altura da árvore, as *Red-Black Trees* utilizam regras de coloração que proporcionam um balanceamento mais flexível, reduzindo o número de rotações necessárias durante inserções e remoções. A comparação entre essas estruturas evidenciou que a escolha depende diretamente do perfil de uso da aplicação, especialmente da frequência relativa entre operações de busca e de atualização da estrutura.

Por fim, analisamos as tabelas *hash*, uma estrutura com abordagem conceitualmente distinta das árvores. Ao empregar funções *hash* para mapear chaves diretamente às posições de memória, as tabelas *hash* permitem acesso próximo de tempo constante, tornando-as extremamente eficientes em cenários em que a ordenação dos dados não é necessária. Foram discutidos o papel das funções *hash*, o problema das colisões e as principais estratégias para tratá-las, além de exemplos práticos de implementação em linguagem C. Assim, esse material reforça que não existe uma estrutura universalmente superior, mas sim estruturas adequadas a diferentes problemas, cabendo ao desenvolvedor compreender suas características para selecionar a solução mais eficiente e coerente com os requisitos do sistema.

## REFERÊNCIAS

ALGOCADEMY BLOG. *Comparing AVL and Red-Black Trees*. 2025. Disponível em: <https://algotcademy.com/blog/balancing-trees-avl-vs-red-black-trees/>. Acesso em: 15 dez. 2025.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. MIT Press, 2009.

CORNELL UNIVERSITY – Departamento de Ciência da Computação. **Balanced Binary Trees Lecture Notes**. Cornell University, 2025. Disponível em: <https://www.cs.cornell.edu/courses/cs3110/2011sp/Lectures/lec10-balanced-trees/balanced-trees.htm>. Acesso em: 14 dez. 2025.

FIVEABLE.ME. **AVL vs. Red-Black Tree Definition**. 2025. Disponível em: <https://fiveable.me/key-terms/data-structures/avl-vs-red-black-tree>. Acesso em: 15 dez. 2025.

GEEKSFORGEEKS. **Introduction to Red-Black Tree**. GeeksforGeeks, 2025. Disponível em: <https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/>. Acesso em: 14 dez. 2025.

GEEKSFORGEEKS. **Red-Black Tree Set 1** (Introduction). GeeksforGeeks, 2025. Disponível em: <https://www.geeksforgeeks.org/red-black-tree-set-1-introduction-2/>. Acesso em: 14 dez. 2025.

RIBEIRO, Pedro. **Balanced Search Trees** – Material de Aula. Universidade do Porto, Departamento de Ciência de Computadores, 2025. Disponível em: <https://www.dcc.fc.up.pt/~pribeiro/aulas/taa1920/balancedsearchtrees.pdf>. Acesso em: 14 dez. 2025.

TERA. **Estrutura de Dados**: O que é? Quais os tipos e exemplos do dia a dia. Tera, 2025. Disponível em: <https://blog.somostera.com/estrutura-de-dados-o-que-%C3%A9>. Acesso em: 15 dez. 2025.